

SEVENTH EDITION

Systems Analysis AND Design

IN A CHANGING WORLD

Chapter 10

JOHN SATZINGER

ROBERT JACKSON

STEPHEN BURD

Approaches to System Development

Chapter 10

Systems Analysis and Design
in a Changing World 6th Ed

Satzinger, Jackson & Burd

Chapter 10 Outline

- The System Development Life Cycle
- Methodologies, Models, Tools, and Techniques
- Agile Development
- The Unified Process, Extreme Programming, and Scrum

Learning Objectives

- Compare the underlying assumptions and uses of a predictive and an adaptive system development life cycle (SDLC)
- Explain what makes up a system development methodology—the SDLC as well as models, tools, and techniques
- Describe the key features of Agile development
- Understand and describe the key features of the Unified Process, Extreme Programming, and Scrum Agile system development methodologies

Overview

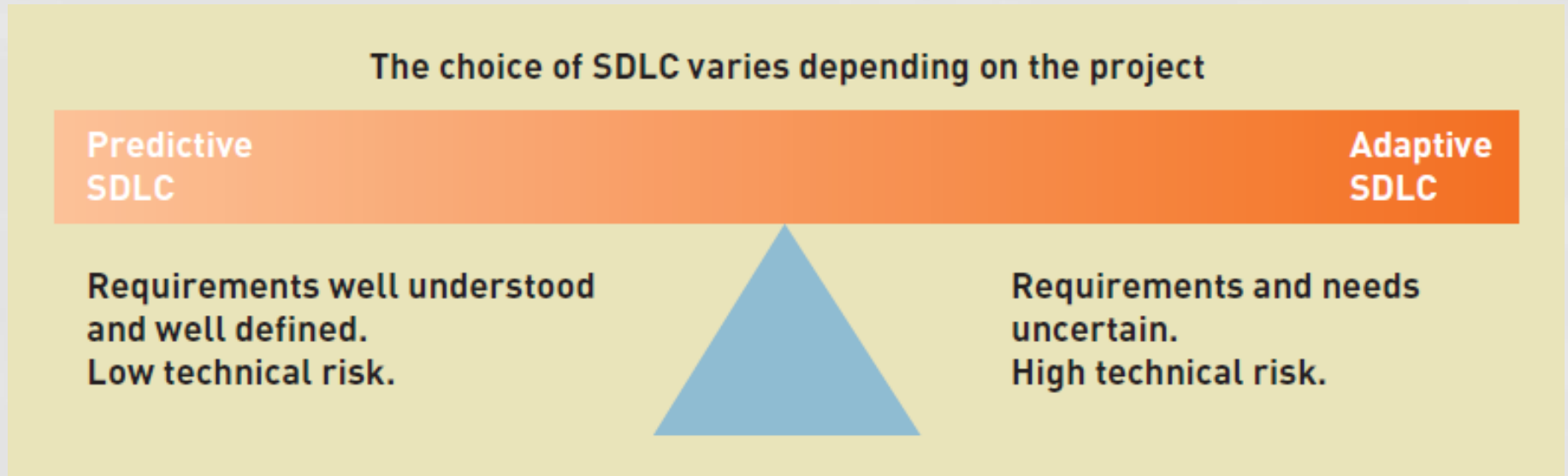
- Chapter 1 demonstrated a system development project that used an iterative and agile system development life cycle (SDLC)
- Later chapters focused on Systems Analysis activities and tasks and some System Design activities and tasks
- Now we return to look at the SDLC and related concepts in more detail
 - Predictive versus Adaptive SDLC variations
 - Activities and Tasks of System Support
 - Models, Methodologies, Tools and Techniques
 - Agile Development
- Specific SDLC Approaches
 - Unified Process (UP)
 - Extreme Programming (XP)
 - Scrum

The System Development Life Cycle (SDLC)

- There are two general approaches to the SDLC
- Predictive Approach to the SDLC
 - Waterfall model
 - Assumes the project can be planned in advance and that the information system can be developed according to the plan
 - Requirements are well understood and/or low technical risk
- Adaptive Approach to the SDLC
 - Iterative model (as see in this text)
 - Assumes the project must be more flexible and adapt to changing needs as the project progresses
 - Requirements and needs are uncertain and/or high technical risk

The System Development Life Cycle (SDLC)

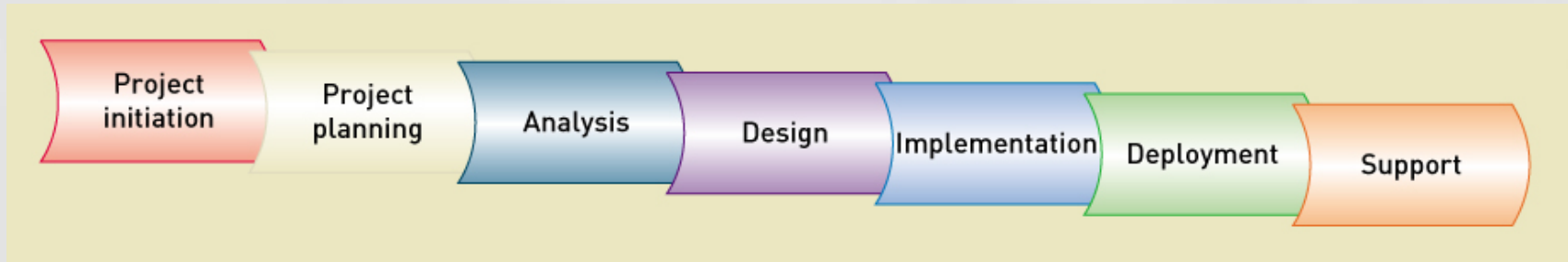
- Most projects fall on a continuum between Predictive and Adaptive



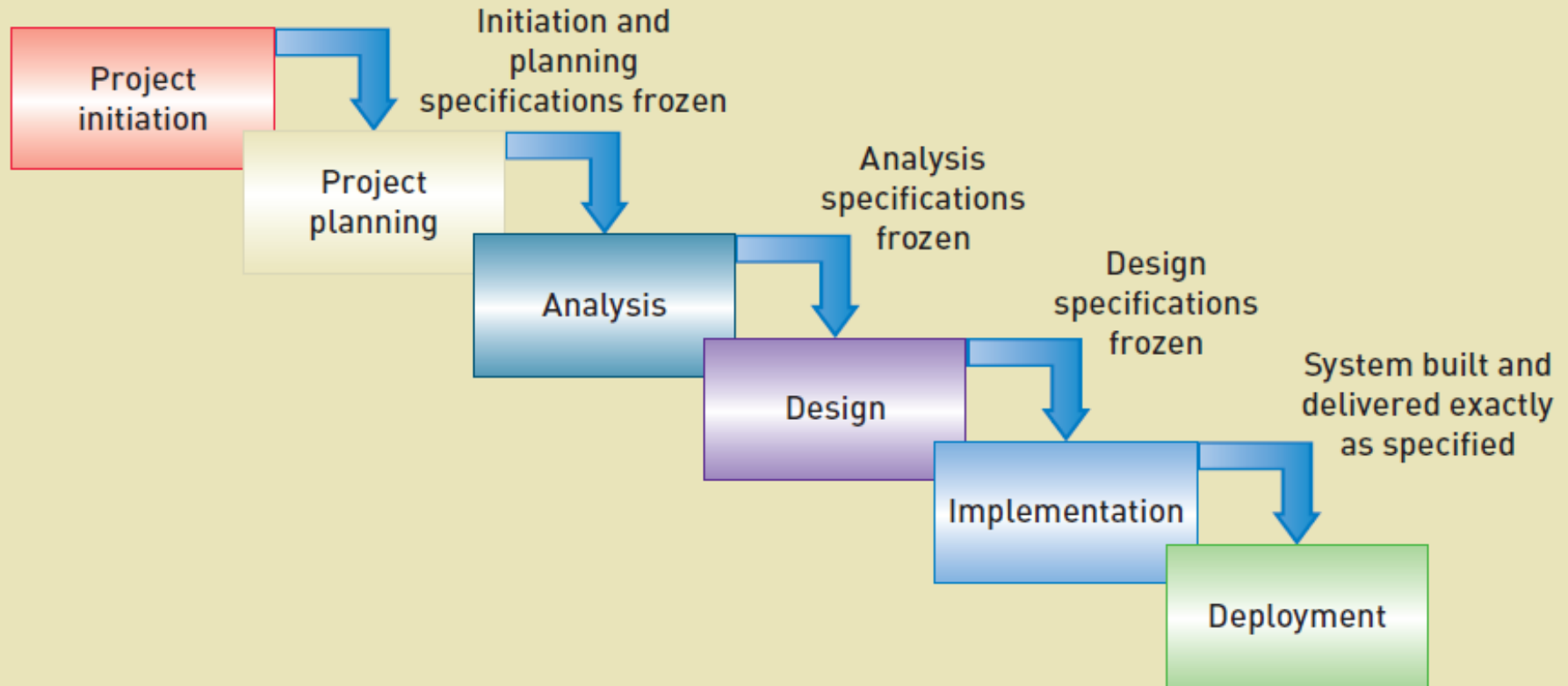
Traditional Predictive SDLC

- Earlier approach based on engineering
- Typically have sequential *Phases*
 - Phases are related groups of development activities, such as planning, analysis, design, implementation, and deployment
- Waterfall model
 - SDLC that assumes phases can be completed sequentially with no overlap or iteration
 - Once one phase is completed, you fall over the waterfall to the next phase, no going back

Tradition IS Development Phases

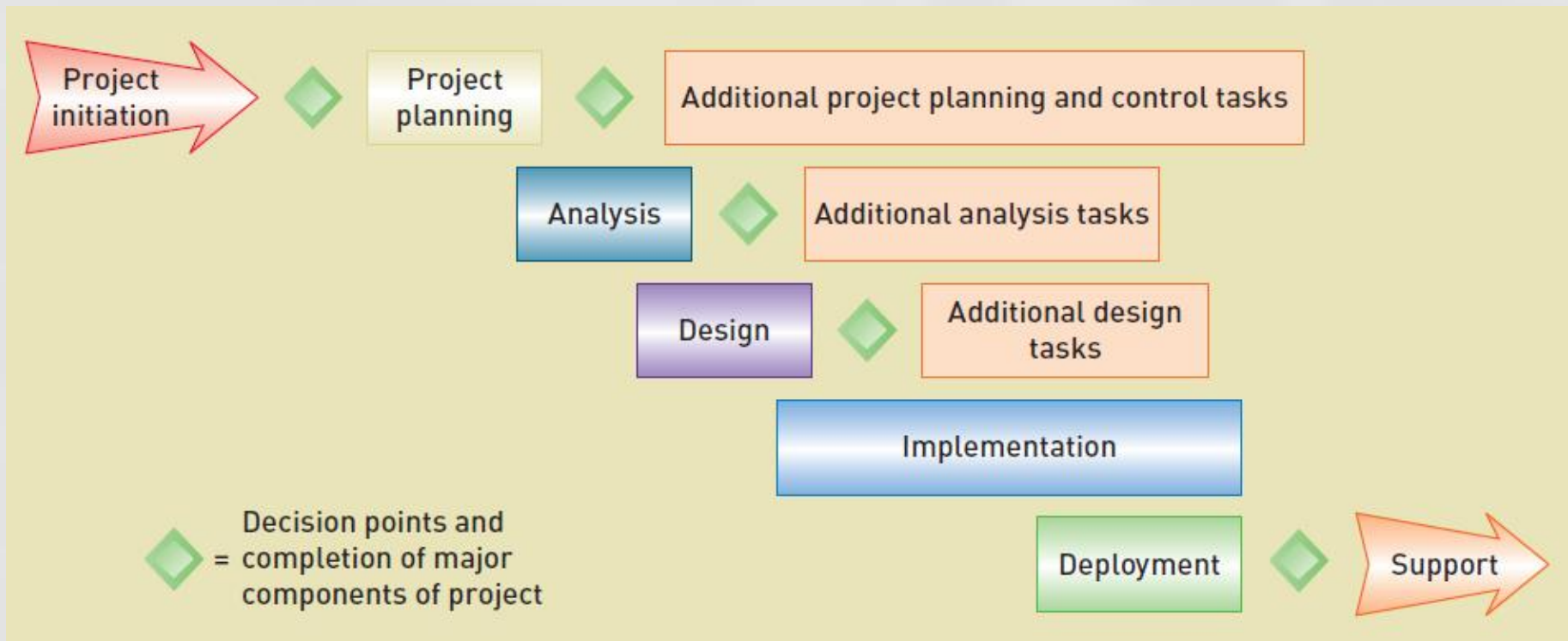


Waterfall Predictive SDLC



Modified Waterfall with Overlapping Phases

- More flexibility, but still assumes predictive planning and sequential phases



Newer Adaptive SDLC

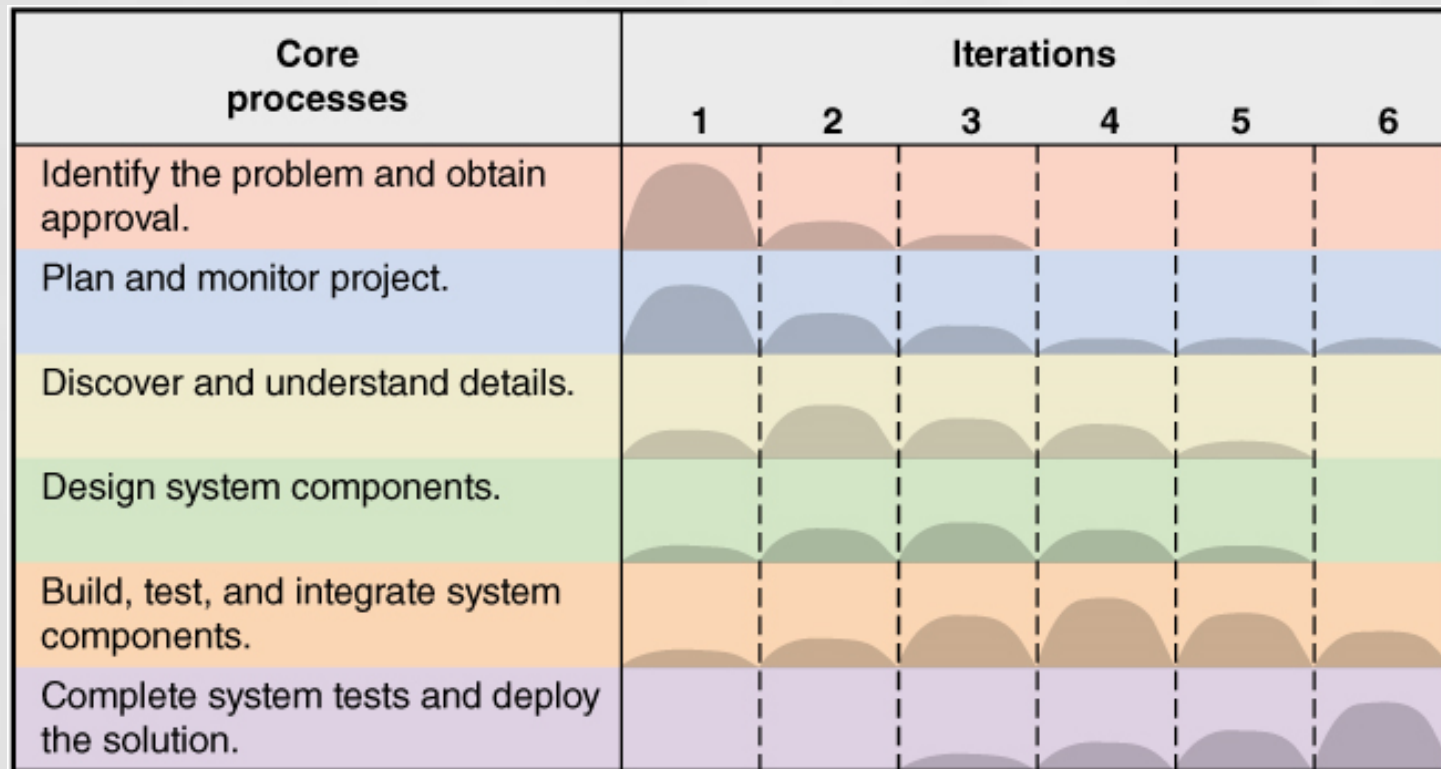
- Emerged in response to increasingly complex requirements and uncertain technological environments
- Always includes iterations where some of design and implementation is done from the beginning
- Many developers claim it is the **only** way to develop information systems
- Many IS managers are still sceptical due to apparent lack of overall plan

Adaptive Approaches

- Incremental development
 - Completes portions of the system in small increments and integrated as the project progresses
 - Sometimes considered “growing” a system
- Walking Skeleton
 - The complete system structure is built first, but with bare-bones functionality

A Generic Adaptive Approach

- Six Core Processes go across iterations
- Multiple Iterations as required



Methodologies, Models, Tools, and Techniques

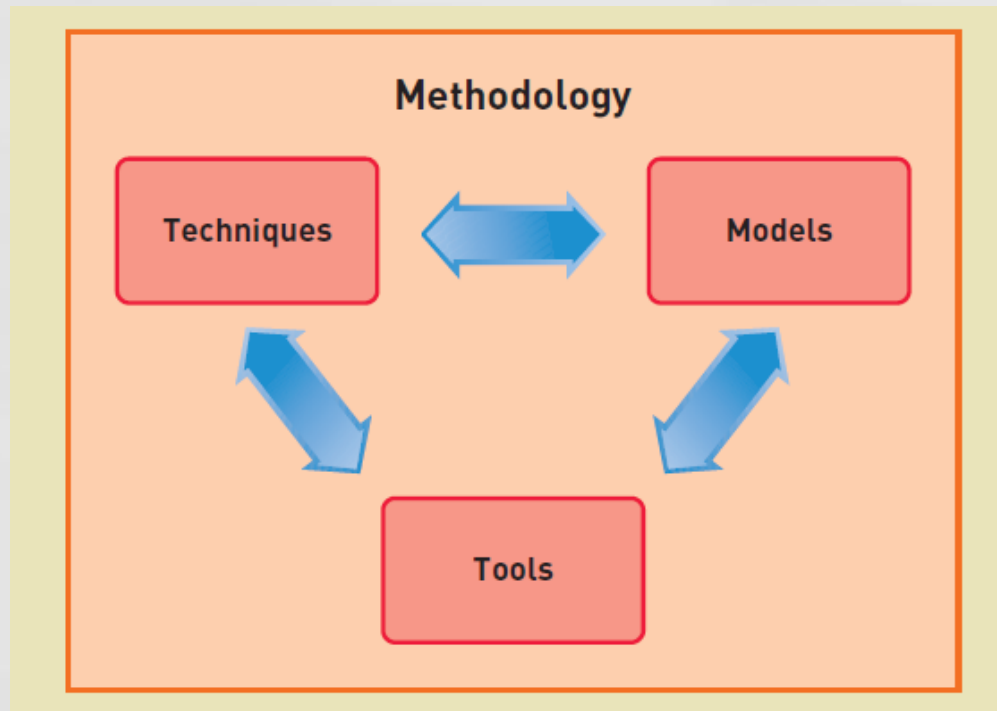
- Methodologies

- Provides guidelines for every facet of system development: What to do when, why and how
- Specifies an SDLC with activities and tasks
- Specifies project planning and project management models and reporting
- Specifies analysis and design models to create
- Specifies implementation and testing techniques
- Specifies deployment and support techniques

- Other term used is *System Development Process*

Methodologies

- A **Methodology** includes a collection of techniques that are used to complete activities and tasks, including modeling, for every aspect of the project



Methodologies, Models, Tools, and Techniques

- Model
 - An abstraction of an important aspect of the real world.
 - Makes it possible to understand a complex concept by focusing only on a relevant part
 - Each model shows a different aspect of the concept
 - Crucial for communicating project information
- In IS, some models are of system components that will be developed
- Other models are used to manage the development process

Methodologies, Models, Tools, and Techniques

Some models of system components

- Use case diagram
- Domain model class diagram
- Design class diagram
- Sequence diagram
- Package diagram
- Screen design template
- Dialog design storyboard
- Entity-relationship diagram (ERD)
- Database schema

Some models used to manage the development process

- Gantt chart
- Organizational hierarchy chart
- Financial analysis models—NPV, payback period
- System development life-cycle model
- Stakeholders list
- Iteration plan

Methodologies, Models, Tools, and Techniques

• Tools

- Software applications that assist developers in creating models or other components required for a project

• Integrated Development Environment (IDE)

- set of tools that work together to provide a comprehensive development environment

• Visual Modeling Tools

- Tools to create graphical models

Methodologies, Models, Tools, and Techniques

- Some typical tools

Project management application
Drawing/graphics application
Word processor/text editor
Visual modeling tool
Integrated development environment (IDE)
Database management application
Reverse-engineering tool
Code generator tool

Methodologies, Models, Tools, and Techniques

● Technique

- A collection of guidelines that help an analyst complete an activity or task
- Learning techniques is the key to having expertise in a field

Strategic planning techniques
Project management techniques
User-interviewing techniques
Data-modeling techniques
Relational database design techniques
Structured programming techniques
Software-testing techniques
Process modeling techniques
Domain modeling techniques
Use case modeling techniques
Object-oriented programming techniques
Architectural design techniques
User-interface design techniques

Agile Development

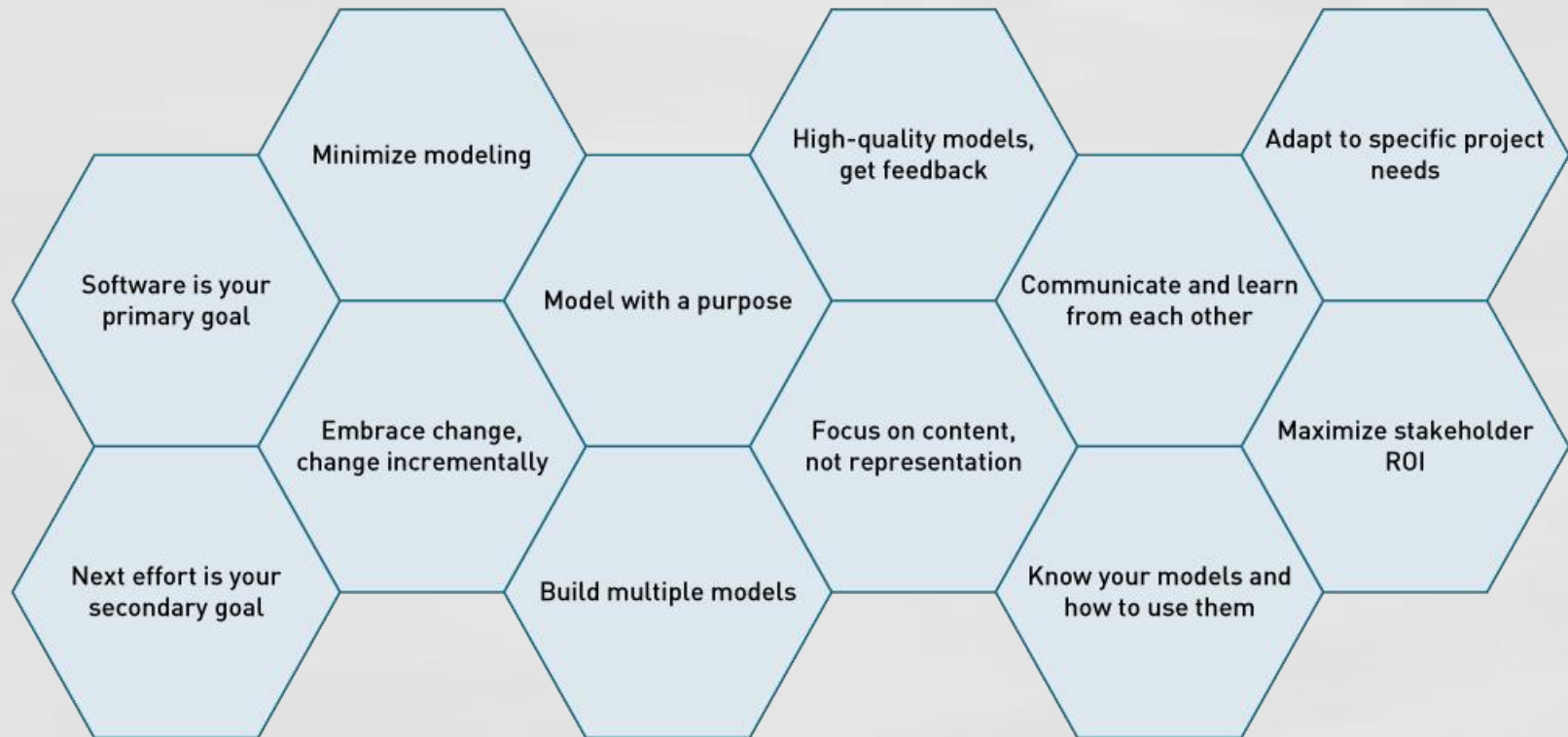
- A guiding philosophy and set of guidelines for developing information systems in an unknown, rapidly changing environment
- Chaordic
 - A term for adaptive projects – chaotic yet ordered

Agile Development

- Agile Values in “Manifesto for Agile Software Development”
 - Value responding to change over following a plan
 - Value individuals and interactions over processes and tools
 - Value working software over comprehensive documentation
 - Value customer collaboration over contract negotiation

Agile Modeling Principles

- Agile Modeling (AM) – 12 Principles
 - A philosophy – build only necessary models that are useful and at the right level of detail



Agile Principles

- Develop software as the primary goal
 - Don't get distracted by documentation or models
- Enable the next effort as your secondary goal
 - Be aware of next step versions or revisions
- Minimize your modeling activity
 - Only build what helps move the project forward
- Embrace change and change incrementally
 - Take small steps that keep you on-track and that can be reversed if necessary

Agile Principles

- Model with a purpose
 - Model to understand
 - Model to communicate
- Build multiple models
 - Look at problems from different perspectives
- Build high-quality models and get feedback
- Focus on content rather than representation
 - Informal hand-drawn models are sometimes okay
 - Always focus on stakeholder needs

Agile Principles

- Learn from each other with open communication
- Know your models and how to use them
- Adapt to specific project needs
- Maximize stakeholder ROI

The Unified Process (UP)

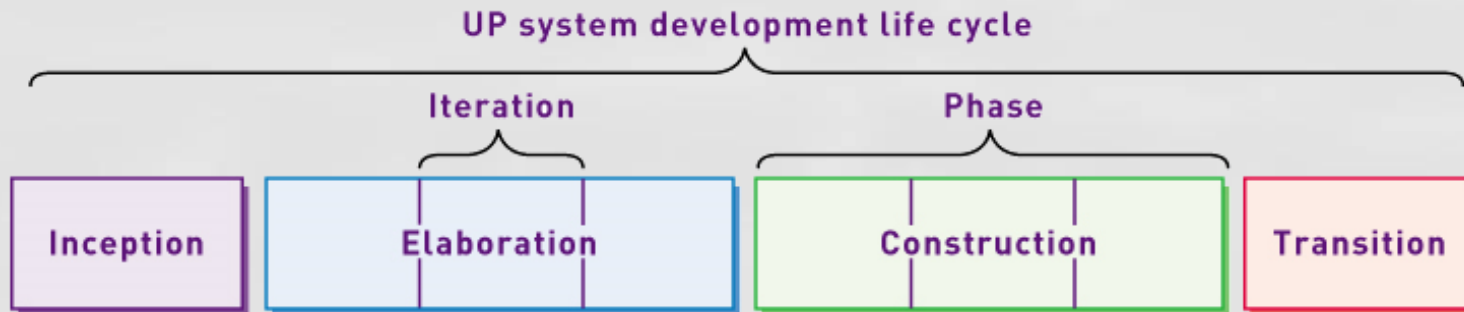
- The UP was the early leader in adaptive approaches
- UP and UML (Unified Modeling Language) were developed together

The Unified Process (UP) – Phases

- UP Phases organize iterations into four primary areas of focus during a project
 - Inception phase – getting the project started
 - Elaboration – understanding the system requirements
 - Construction – building the system
 - Transitions – preparing for and moving to deploying the new system

Unified Process (UP) – Phases

- UP SDLC divides iterations into four phases



Unified Process (UP) – Phases

- Objectives of each phase

UP Phase	Objective
Inception	Develop an approximate vision of the system, make the business case, define the scope, and produce rough estimates for cost and schedule.
Elaboration	Define the vision, identify and describe all requirements, finalize the scope, design and implement the core architecture and functions, resolve high risks, and produce realistic estimates for cost and schedule.
Construction	Iteratively implement the remaining lower-risk, predictable, and easier elements and prepare for deployment.
Transition	Complete the beta test and deployment so users have a working system and are ready to benefit as expected.

Unified Process – Disciplines

- A set of functionally related development activities
 - Each discipline are all the activities related to achieving one objective in the development project
 - Two types of disciplines
 - Development disciplines
 - Management – planning and control disciplines

Unified Process – Disciplines

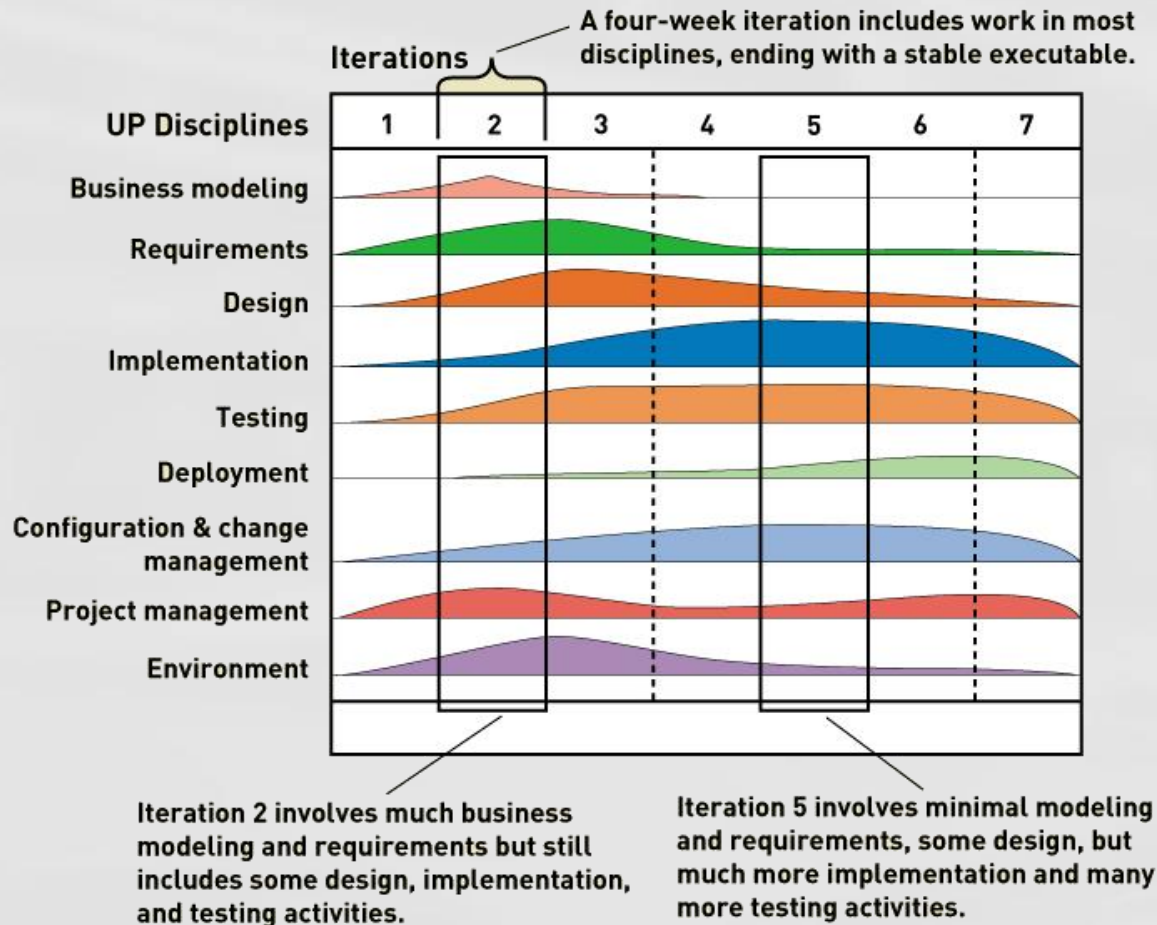
- Development Disciplines
 - Business modeling
 - Requirements
 - Design
 - Implementation
 - Testing
 - Deployment

Unified Process – Disciplines

- Planning and Control disciplines
 - Configuration and change management
 - Project management
 - Environment

Unified Process – Disciplines

- Disciplines are used across all iterations



Extreme Programming (XP)

- Takes the best practices of software development and extends them “to the extreme”
 - Focus intensely on proven industry practices
 - Combine them in unique ways to get better results
- XP Core Values
 - Communication
 - Simplicity
 - Feedback
 - Courage

Extreme Programming (XP)

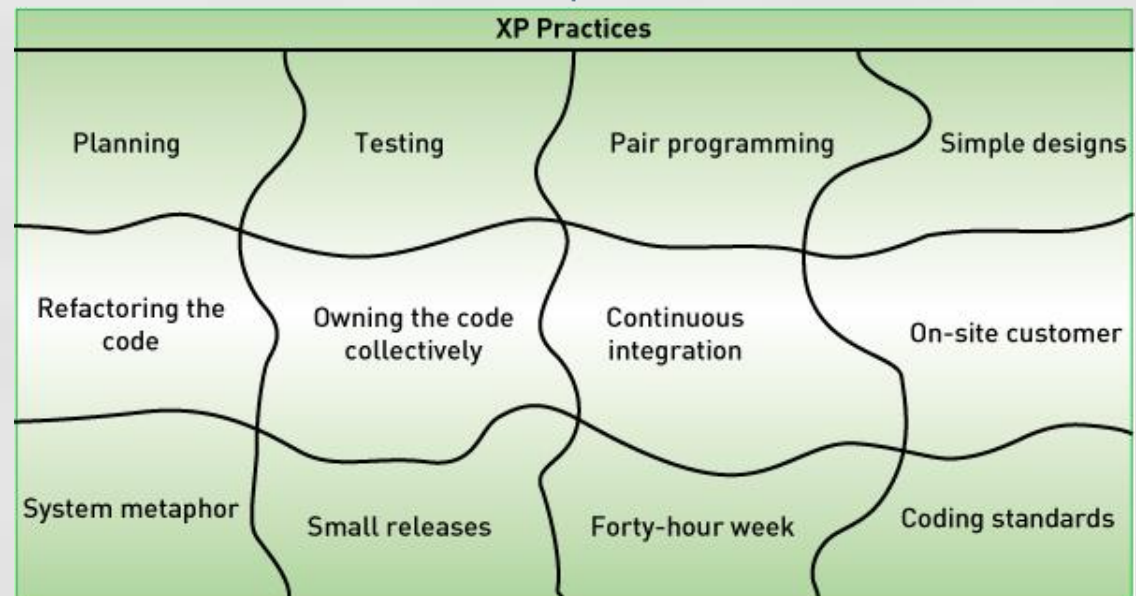
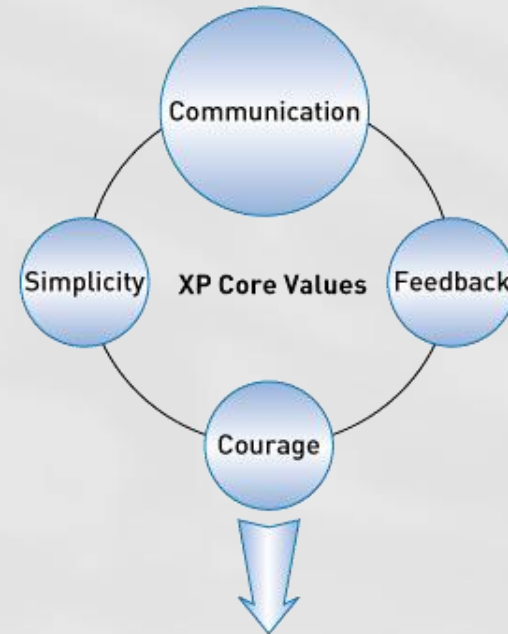
● XP Practices

- Planning – based on user stories
- Testing – thorough testing at every step
- Pair Programming – watch, inspect, trade off
- Simple Designs – Agile modeling principles
- Refactoring – redo and cleanup as you go
- Owning the code collectively – egoless development, anyone can review and improve code

Extreme Programming (XP)

- XP Practices (cont)
 - Continuous integration – grow the software continuously
 - On-site customer – get sign-off as you go
 - System metaphor – what should the final system look like
 - Small releases – turn over to user frequently
 - Forty-hour work week – don't overload the developers
 - Coding standards – follow standards for code

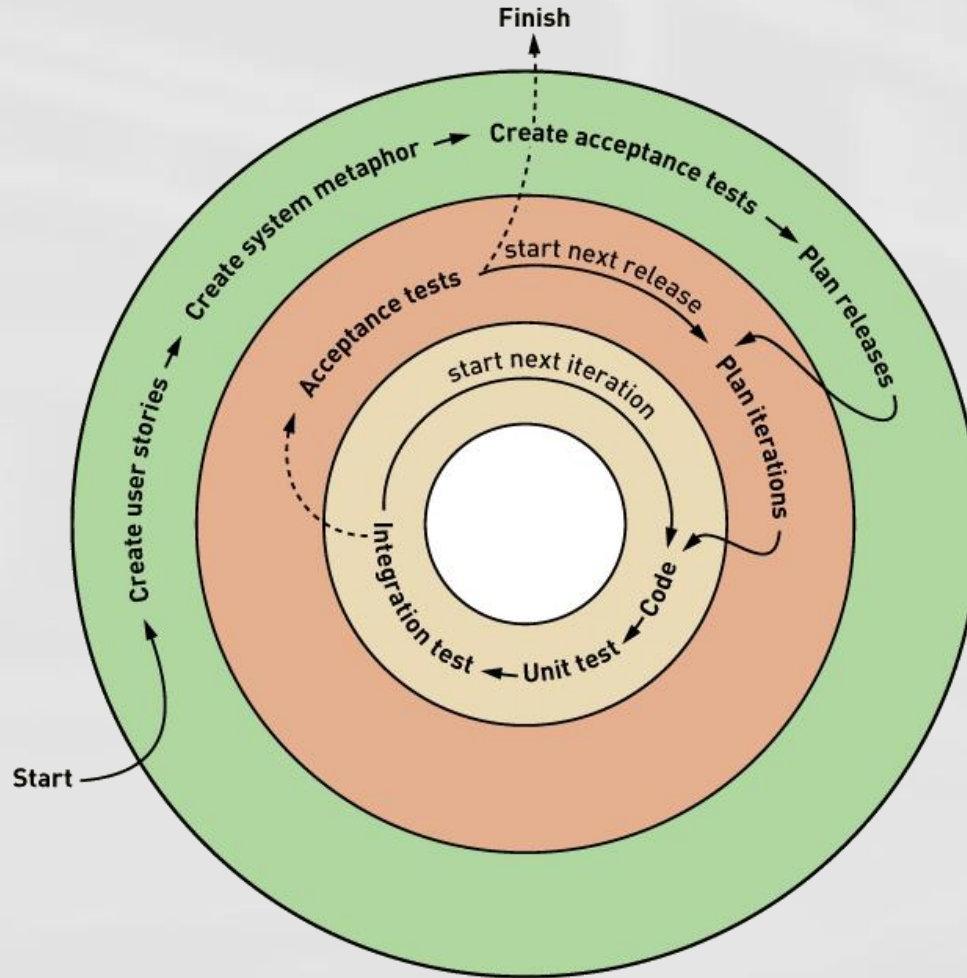
XP Core Values and Practices



XP Project Approach

- Three level approach (three rings)
 - Outside ring – create user stories and define acceptance tests
 - Middle ring – conduct tests and do overall planning
 - Inside ring – iterations of coding and testing

XP Project Approach

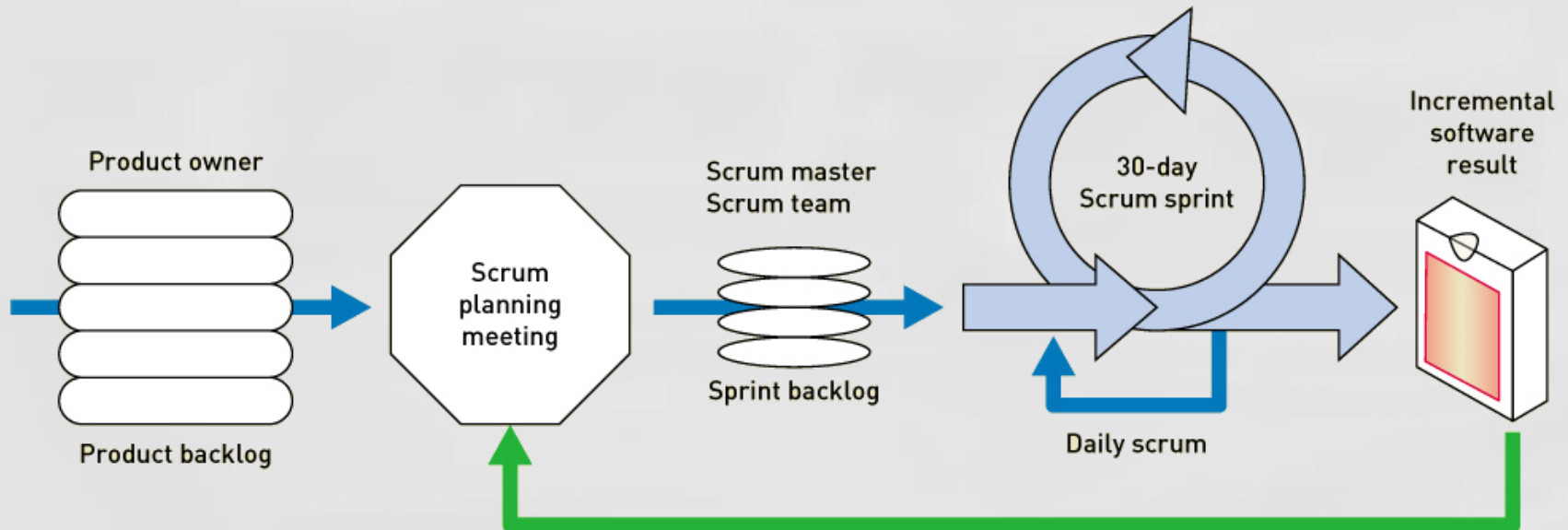


SCRUM

- Combination of principles of Rugby and Agile
 - Intense effort involving the entire team for a defined period of time
- Product backlog
 - Prioritized list of user requirements
- Product owner
 - The client stakeholder who controls backlog
- Scrum master
 - Scrum project manager

Scrum Sprint

- Scrum sprint
 - A time-controlled mini-project to implement part of the system



Scrum Practices

- Scope of each sprint is frozen (but can be reduced if necessary)
- Time period is kept constant
- Daily Scrum meeting
 - What have you done since the last daily Scrum (during the last 24 hours)?
 - What will you do by the next daily Scrum?
 - What kept you or is keeping you from completing your work?

Summary

- This chapter covers approaches to system development in more detail
- There are two approaches to the SDLC: Predictive and Adaptive
- A predictive SDLC, also known as the waterfall model, is used when it is possible to plan the project completely in advance
- An Adaptive SDLC, which uses iteration, is used when the requirements are less certain and the project will need to react to changes

Summary (continued)

- All new System development project use a methodology (or development process) and many are available. A methodology includes an SDLC and tools, techniques, and models
- Agile development is the current trend in system development
- Unified Process is a formal iterative approach which uses UML models (with Agile philosophy) and UP disciplines
- Extreme Programming (XP) is an iterative approach which takes good practices to the extreme
- Scrum is an iterative approach using a Scrum Sprint